PROJECT BRIEF

Proofed

Integrated ASR + post-processing model, from speech straight to clean, proofed text

Daniel Rosehill · 2026-10-24

Defined by	Daniel Rosehill, voice note recorded 2026-10-24 (24:40 long)
Prepared by	Claude Code · Opus 5.5 (claude-opus-5-5)
Generated	2026-10-24 14:20:18 +03:00
Transcription	AssemblyAI universal (transcript fbc93846-95b4-4d0b-8121-e45a285ccebb), lightly edited by Claude
Repository	github.com/danielrosehill/Proofed @ 3806576

Summary

Proofed fine-tunes a stock, phone-sized speech-recognition model (Parakeet TDT 0.6B v2) only on cleanup tasks, so it writes clean text in one pass: no fillers, repeats or false starts, numbers as digits, self-corrections applied, asides left out, paragraphs, and whispered speech handled. Domain vocabulary is out of scope. The first phase is research: Parakeet's paragraph token, prior art, candidate base models and feasibility.

Contents

1 Overview 3
- 1.1 What it is 3
- 1.2 The problem 3
- 1.3 The objective 3
- 1.4 Scope 3
- 1.5 Approach 3
- 1.6 Background 3
- 1.7 Open questions from the note 4

2 Development plan 4
- 2.1 What it is 4
- 2.2 Why 4
- 2.3 Scope 4
- 2.4 Approach 5
- 2.5 Phases and exit tests 6
- 2.6 Open questions (need Daniel) 6
- 2.7 Risks 6

3 The voice note, lightly edited 7
- 3.1 Background: voice as the way in (00:00) 7
- 3.2 The basic contract of ASR (01:58) 7
- 3.3 From post-processing prompts to audio-native models (02:52) 7

3.4 Why that isn’t the whole answer: size (04:55) 7
3.5 Dedicated post-processing models (05:53) 8
3.6 What I’ve been fine-tuning (06:11) 8
3.7 One model across devices (07:52) 8
3.8 How the task list grew (08:37) 8
3.9 Combining domain fine-tunes didn’t hurt (10:58) 9
3.10 The idea: Proofed (12:34) 9
3.11 Layering (13:55) 9
3.12 Where the line is (15:06) 9
3.13 Paragraph inference (16:19) 10
3.14 The experimental one: non-directed speech (17:04) 10
3.15 Keep tasks narrow and modular (17:54) 10
3.16 The objective (19:27) 10
3.17 The prototype (20:25) 10
3.18 Whispered sentences (21:03) 10
3.19 Data and prior art (22:00) 11
3.20 Stages (22:47) 11
3.21 Open question: English-only? (23:39) 11
3.22 Correction: domain vocab is out of scope (23:54) 11
3.23 Corrections 11

1 Overview

1.1 What it is

Proofed is a speech-recognition model that writes clean text directly. It starts from a stock, phone-sized ASR model (NVIDIA Parakeet TDT 0.6B v2) and fine-tunes it only on post-processing tasks, so the transcript comes out already proofed, in a single forward pass, with no LLM step afterwards (12:34, 19:27).

1.2 The problem

The basic contract of ASR is narrow: render sounds as words. Everything beyond that (punctuation, sentence boundaries, removing “um”) is an addition someone chose to build (01:58). Today those additions come from a second model: a general LLM with instructions, or a dedicated text post-processor such as Superwhisper’s S1-mini (05:53). Audio-native multimodal models can do any transformation but are far too big for a phone and unusable for dictation even on a desktop GPU with 12 GB of VRAM (04:55). Regex cleanup after inference works for some of it; the aim is to fix it at source (16:03).

1.3 The objective

Find out to what extent a stock ASR model can be fine-tuned to bake in a set of post-processing transformations (12:34). Same result for the user as ASR plus cleanup, from one small model. Published on Hugging Face as an experimental prototype (20:25).

1.4 Scope

In: transformations that are right for every user every time (15:17):

- removing fillers (“um”, “uh”), repeated words, false starts and unfinished sentences
- numbers written as digits
- applying self-corrections and editing instructions (“Haifa, I mean Tiberias”; “scratch that”)
- paragraph breaks
- leaving out speech aimed at someone else in the room (the experimental task)
- whispered speech, so dictation works when others are asleep (21:03)

Out: any domain vocabulary, including code-switching and tech terms (23:54), and anything use-specific, such as formatting every output as an email (15:17).

1.5 Approach

Take Daniel’s existing “kitchen sink” recordings, already stratified by task type, keep only the post-processing tasks, and tier them as core, ambitious and experimental (20:25). Define each task narrowly so each can be evaluated alone, since some may not train well at small scale (17:54). Later, test a layered approach: fine-tune Proofed again with a domain-vocabulary layer and compare with a one-shot model (13:55).

1.6 Background

Proofed grows out of Daniel’s Parakeet and Nemotron fine-tunes for English-Hebrew code-switching and tech vocabulary (06:11). While adding number rendering and disfluency removal to that dataset, he noticed the work had crossed from domain-vocabulary fine-tuning into post-processing (12:34). An earlier finding encouraged him: combining two domain fine-tunes in one “kitchen sink” dataset did not degrade either (10:58).

1.7 Open questions from the note

- Does Parakeet have a token for a blank line? If not, paragraphs need a different base model or a decoder change (16:19).
- Can one model hold both an ASR pass and a post-processing pass internally (19:27)?
- Can the model be English-only (23:39)? With domain vocabulary out of scope, probably yes.
- Which existing attempts at a unified ASR and post-processing model are there (22:00)?

2 Development plan

Status 2026-10-24: planning. No data split, no training run, no model yet. Source: voice note 2026-10-24 (cited as `vn mm:ss`). Where this plan and the note disagree, the note wins; record the decision here.

Decided = said by Daniel in the note. **Inferred** = this plan's reading, to confirm.

2.1 What it is

An ASR fine-tune that writes clean text directly: a stock ASR base model (Parakeet first) trained only on post-processing tasks, so that the transcript comes out without disfluencies, with numbers as digits, without asides to other people, and (if the base model allows) with paragraphs. One forward pass, no second LLM step. (Decided, vn 12:34, 19:27.)

It is a research prototype: the question is *to what extent a stock ASR model can be fine-tuned to bake in a series of post-processing transformations* (vn 12:34). Published on Hugging Face (vn 20:25).

2.2 Why

- Audio-native LLMs (Gemini with a system prompt) can do any transformation, but they are too big for phones and unusable for dictation even on a 12 GB desktop GPU (vn 04:55).
- Dedicated text post-processors (S1-mini by Superwhisper) still need a second model (vn 05:53). Daniel's own text-side attempts: `danielrosehill/ASR-Post-Processor-Model` (planning, S1-mini LoRA) and `danielrosehill/ASR-Paragraph-Breaks` (Gemma 3 270M, break F1 0.78, released 2026-10-21).
- Regex post-inference cleanup works for some of this; the aim is to fix it at source (vn 16:03).
- The Hebrish kitchen-sink run showed that combining training domains did not degrade either (vn 10:58). Inferred: that is some evidence task subsets can be stacked.

2.3 Scope

In (decided): post-processing transformations that are right for every user, every time (vn 15:17). Narrow, modular tasks, each measurable on its own, because some may not train at small scale (vn 17:54).

Out (decided): all domain vocabulary, including code-switching, tech vocab, names (vn 23:54). Anything use-specific, such as formatting output as an email (vn 15:17).

Tiers. Daniel names core, ambitious and experimental tiers (vn 20:25) but assigns only some tasks. Proposed mapping onto the existing task categories (`audio-dictation/Hebrish-App/data/training-categories.json`, 2026-10-21):

Tier	Category id	Task	Basis
core	`disfluency`	drop uh / um / eh	decided, vn 09:55, 15:17
core	`repetitions`	drop restart repeats (“orange, orange juice”)	decided, vn 17:54
core	`false-starts`	drop abandoned starts and unfinished sentences	decided, vn 17:54
core	`numerics`	numbers, dates, units in digits	decided, vn 09:55
core	`currency`	spoken amounts as typed (50 NIS)	inferred: general formatting
ambitious	`self-corrections`	“Haifa, I mean Tiberias” → Tiberias; “scratch that”	inferred tier; task decided, vn 18:32
ambitious	`paragraphs`	paragraph breaks	decided task, vn 16:19; tier inferred, blocked by tokenizer (below)
experimental	`asides`	drop speech aimed at someone else	decided, vn 17:04
robustness	`whispered`	whispered speech	decided, must be included, vn 21:03
open	`web-addresses`, `spelled-out`, `acronyms`, `expanded-acronyms`	written-form conventions	unclear whether “post-processing” or “domain”; ask
out	`singing`, `weird-voices`, `name-cipher`	other experiments	inferred out: not mentioned

Plus plain-English replay (`general`, LibriSpeech-PC) so the base does not forget; ratio per `ai-models/ASR-Dataset-Blending` (start ~10%).

2.4 Approach

1. **Base model:** `nvidia/parakeet-tdt-0.6b-v2` (English, punctuated and cased). Daniel prefers to stay on Parakeet (vn 22:47). Survey alternatives anyway (task in `tasks.md`).
2. **Data:** from the kitchen-sink recordings (`danielrosehill/Read-Aloud-Voice-Sentences` and the Hebrish blends, e.g. `Hebrish/Parakeet-V2-Blend-All`), keep only the post-processing categories above, stratified by category. Same term-disjoint train/test split as the Hebrish repos. Measure minutes per category first: Daniel expects some need more recording (vn 22:00).
3. **Training:** NeMo fine-tune, as in `audio-dictation/English-Hebrew-Codeswitching-Parakeet-Finetune` (Modal GPU).
4. **Evaluation:** per category, against the clean target: task success (filler gone, digits right, aside removed, correction applied) plus WER on the rest of the sentence, and WER on `general` and

LibriSpeech-PC for regression. Compare with stock Parakeet, and stock Parakeet + regex + `ASR-Paragraph-Breaks` as the two-model baseline.

5. **Layering (later):** fine-tune Proofed again with a domain-vocab layer (tech, code-switching) and compare with the one-shot kitchen-sink model (vn 13:55).

2.5 Phases and exit tests

Phase	Work	Exit test
0. Brief	this repo, cleaned transcript, plan, tasks	done 2026-10-24
1. Research	prior art (unified ASR + post-processing models), ASR models with a paragraph/ newline token, feasibility of training these tasks into an ASR decoder, candidate base models	`docs/research.md` with sources, dates, and a confirmed/inferred status per claim
2. Data audit	minutes and takes per in-scope category; gaps	table in `docs/data.md`; every core category ≥ the minimum Daniel sets
3. Spec	final task list and tiers, targets, metrics, blend ratios	`docs/spec.md` agreed
4. Prototype	first fine-tune on core + whispered	per-category eval vs stock Parakeet; no regression over 1 pt WER on `general`
5. Extend	ambitious and experimental tasks one at a time	each task's eval, kept or dropped with a reason
6. Publish	HF model, card, eval table	model card live

2.6 Open questions (need Daniel)

1. **English-only base?** Raised at vn 23:39. With domain vocab out of scope (vn 23:54), inferred answer: yes, the English-only Parakeet v2 is fine; code-switching comes back only in a later layer, and that layer's base question can wait. Confirm.
2. **Paragraphs.** Parakeet's tokenizer may have no newline token (vn 16:19, "if I'm not mistaken"); to verify in phase 1. If not: add a token and resize the decoder, change base model, or leave paragraphs to `ASR-Paragraph-Breaks`?
3. **Tiers.** Confirm the table above, especially the "open" row.
4. **One model or two passes inside one?** vn 19:27 asks whether one model can hold an ASR pass and a post-processing pass. Treated as a research question in phase 1, not a design commitment.
5. **Recording budget.** How many more minutes per thin category is Daniel willing to record?

2.7 Risks

- Tasks such as asides and self-corrections need context the ASR decoder may not model well at this size; they may not train (Daniel expects some won't, vn 17:54).
- Cleaning in the decoder can over-delete real words; the eval must measure WER on the kept text, not only task success.
- Read-aloud disfluency data is acted, not natural; it may not transfer to real dictation.

3 The voice note, lightly edited

Voice note recorded 2026-10-24 by Daniel Rosehill (24:40). Cleaned from `transcript-raw.txt` (AssemblyAI universal, unedited). Fillers, false starts and repetitions are removed and obvious mishearings fixed (listed under Corrections at the end). The words, order and points are the speaker's. Timestamps are `mm:ss` into `voice-note.mp3`.

3.1 Background: voice as the way in (00:00)

This voice note was recorded on 24 October 2026. It's context for a project that builds on what I've been working on for the past week, and really on a couple of preceding years of using voice dictation all day, every day, across different devices, as my preferred way of entering text into a computer: trying out my own fine-tunes, trying out different apps, seeing what works and what breaks, and seeing how the available models have changed and are changing. That goes from the amazing Whisper, an outstanding model, to the directions people have been taking for that final push in ASR and voice dictation utility: from "this is very impressive, very accurate, but I still need to put effort into this before I send it [unclear: "to the ladder"]" to something you don't have to touch.

I think this is a very exciting area right now, because what I think is essential is the action taken on the back of voice notes. If it stops at the transcription, a huge amount of the potential utility of dictation and voice as an interface for getting things done is lost, maybe even most of it. With easy-to-use connectors for email, calendar, contacts, you name it, available in apps like ChatGPT, the complication and bugginess of MCP is no longer something that most people, or anyone for that matter, has to wade through just to get these things done.

3.2 The basic contract of ASR (01:58)

The basic contract of ASR is extremely limited in scope: taking what people say, phonetics, sounds, and rendering that into words. What's underappreciated is that everything bolted onto that very bare level of scope is a decision and an addition. It's work that someone undertook to go beyond the basic contract. Someone said: hang on, we need sentences, we need punctuation, we need to detect sentence boundaries, and so on.

3.3 From post-processing prompts to audio-native models (02:52)

There have been a couple of changes I've been looking at for my own tools. Over the course of building these out, I developed a bank of what I call text transformation prompts. Initially that was the classic post-processing step for ASR plus LLM: the second step gets the ASR text back and says, restructure this text as an email, a formal email, a casual email, a shopping list, a task list. When I was drafting these, I realised the number of permutations was enormous, since you can build them one on top of the other.

I did create an app for my own use that did exactly that. Under the hood it was prompt concatenation. I had boxes for "make this more formal", "less formal", "polite business email"; some of them instructing the output format, some of them instructing the style. When the multimodal models started appearing, the Gemini models that could process audio tokens and take a system prompt, it struck me that this is gigantic for dictation, because I can create basically any instruction, even "translate this into formal Hebrew". In other words, I can mop up even translation tools.

My thought at that point was: what's the point of having ASR and cleanup models if you have these? I've been thinking about that a lot since. Is that the future?

3.4 Why that isn't the whole answer: size (04:55)

One of the negatives of that line of prediction is that, to the best of my knowledge, there aren't very many small models that can do audio. On-device ASR has quickly matured, through excellent models

from Moonshine and NVIDIA. The multimodal audio models are still just too big for most people. I don't even know what phone could run them; they're not practical for the most part. Even on the desktop with 12 GB of VRAM, I haven't found one that runs in any way that's usable for dictation, which is a separate concern.

I'm tracing the full evolution of the field as I see it, to explain where I see this idea fitting in.

3.5 Dedicated post-processing models (05:53)

Then there's a model called S1-mini by Superwhisper, which is a dedicated post-processing model. In other words, instead of taking a general-purpose LLM and adding your own instructions to an instruction-tuned model, it's something actually trained to expect ASR text, with a defined system prompt honed on this specific task. That's a good idea.

3.6 What I've been fine-tuning (06:11)

In the last couple of weeks I've been fine-tuning Parakeet and Nemotron on a code-switching task. I live in Israel, and when I began using Whisper very ardently, one of the first things that jumped out at me was that it was going to fail on all the Hebrewisms, as it were: "I'm going to the shuk and going to Bituach Leumi after this." So my first fine-tune for Whisper was code-switching.

Then, again just because I'm using these things day in, day out, I noticed that tech vocab and new brand names are a problem. That makes sense, because they're still fairly niche, and beyond being niche they change very fast. I might begin talking about S1-mini from Superwhisper a lot, or ROCm, as in ROCm for the AMD stuff.

For this round of fine-tuning I moved from Whisper to Parakeet, because I found that stock Parakeet was far, far superior on my phone, which is what I've happened to be using for the past few months rather than my desktop.

3.7 One model across devices (07:52)

I'm personally in favour of using one model across different devices. You might have a model that's the best choice for GPU acceleration on one machine. But this is my philosophy: if you're working towards extremely high accuracy and exactly the type of output you want, there's more to be said for knowing exactly the limits, the pros and the strengths of one model than for using a couple of them. It gets hard, it gets confusing: which model doesn't handle the disfluencies, which does. So I'm going all in on Parakeet and the NVIDIA models generally.

3.8 How the task list grew (08:37)

For the fine-tune I started training last week, after I moved over, I didn't even consider using my previous Whisper dataset, because I thought it was a bit old. So I started recording new code-switched sentences. And while doing that I thought: if I'm going to sit down to do this… It's worth it when you start seeing the evals move in the right direction, but it's kind of weird to be sitting in bed reading these random sentences, and it's tedious.

So I thought, if I'm going through this process again, I may as well think of everything I want: everything that's wrong with this model as I see it, or everything it does poorly. After code-switching I added recognition for tech vocab, and then I asked what else would be nice for this model to do, and began drawing up a list as I transcribed with it. These are things I've just observed; it's not as strong as Whisper in some ways, maybe because it's a smaller model.

One was number rendering. The classic example: "the year is 2026" could be rendered in spelled-out numbers, but no one writes a date that way, so it should be 2026 in numerals. Then just disfluency removal: Parakeet very faithfully renders all the ums and ahs.

As I built up these tasks and datasets and trained on them, I realised I had crossed a juncture, or a division.

3.9 Combining domain fine-tunes didn't hurt (10:58)

My first experiment was to see whether two sets of fine-tuned vocab would degrade each other's accuracy, which my evals said they didn't. That was very encouraging, because I try to balance my desire to open source with the time I have to dedicate to a project. In an ideal world I'd maintain a tech one, because that could be useful for a lot of people, and Parakeet fine-tuned for code-switching plus tech for my own use. But in the real world, to the extent that I can afford to spend time on my open-source projects, it's biased towards the tools I'm actually going to use and benefit from most myself, which is the all-in-one.

I called it the kitchen-sink training dataset: tech plus Hebrew. That was a surprising result. I assumed, and I'm not sure why, it just seemed common sense, that if you distribute the weights you're increasing recognition for across tech vocab and Hebrew, the jack of all trades is going to be master of none. And even if the eval had shown a 5% decrease in accuracy on both domain datasets, I would have said that's a good trade-off, because the most useful model for me is the one that does code-switching and tech without overfitting on either niche domain.

3.10 The idea: Proofed (12:34)

Coming back to the thought I was finishing earlier: when I began expanding the dataset, because who wants to do this multiple times, I realised that the first two datasets are training on the classic ASR fine-tuning task of domain vocabulary. The second group, which I'm building out now, crossed over once I added disfluency removal and number rendition: we're now integrating a post-processing step into the model instead of doing it separately.

So what I want to train with this model, which I'm calling Proofed (I couldn't think of a better name right now, but it's a decent one), is an ASR model trained only on a base ASR model plus cleanup. I'm not sure this is actually the fine-tuning methodology that makes sense, but it would nevertheless be very worthwhile, just to see how well this works in isolation. This is the isolated model, to test to what extent a stock ASR model can be fine-tuned to bake in a series of post-processing transformations.

3.11 Layering (13:55)

Then, can we take a layering approach and fine-tune the fine-tune to add the second layer of needs, like the tech vocab? In other words, you take a model like Parakeet; Proofed is Parakeet plus fine-tuning for post-processing and formatting; and then you build on top of it. It's a higher workload from a training perspective, since the easiest way to get to any result is one dataset and one training run, but it could be a very elegant way to do it.

What I want to do for this one, maybe not for my own use but as an experiment, is throw in as many of the datasets I've been creating specifically for post-processing as I can. I've deliberately created a mixture, starting with what I'd call the core ones.

3.12 Where the line is (15:06)

There is a limit, a definite line you don't want to cross. If you're going to bake in deterministic post-processing, you want it to stay general. (And maybe this is where it could pair with a second pass.) If, for argument's sake, Proofed were trained to format every output as an email, it would be useless for anyone who wants it for any other purpose. So the set of post-processing transformations I'm trying to train on are the ones that make sense for everyone. For my use, I can't think of a single time I've wanted disfluencies like "um" to appear in a transcript. They're always unwanted, and that's the classic example of the type of thing I want to bake in here.

Those can be handled at the regex level, meaning post-inference. But the idea here is to make that unnecessary in the first place, to tackle it at source.

3.13 Paragraph inference (16:19)

Another one I feel strongly about is paragraph inference. I find it's very overlooked, in ASR and even in post-processing: sentence boundary detection is one thing, but adding paragraph breaks is another. This is a bit of a problem with Parakeet, because if I'm not mistaken the core model doesn't have a token for a blank line, which makes training for paragraph inference impossible in the raw model. So I might need to find a different base model, or add that into the decoder.

3.14 The experimental one: non-directed speech (17:04)

Those are the standard ones on my list. The experimental one, as I call it, is non-directed speech, which is kind of a reasoning task. For example, you're recording and you say: "So the API documents are here now… yeah, just leave it by the door." The idea is to infer that "just leave it by the door" wasn't meant for the transcription: the speaker was talking to somebody else, and it shouldn't be included. That's an edge case: if it worked, it might be very useful; if it didn't, not so much.

3.15 Keep tasks narrow and modular (17:54)

I think it makes sense to define these tasks very narrowly and modularly, because in seeing what actually works, it could be that certain tasks just can't be trained reliably at a small scale and some can.

The other common disfluency is repeated words: "I'm putting the orange, orange juice in the fridge." You repeat words when you're distracted. False starts are another: "And going to the… I'm going to the cinema." Another example is sentences you never finish: "I think I had an idea… I want to go to the movies now."

Another is inferred editing instructions. A self-correction: "I need to go to Haifa today. I mean Tiberias." Whereas an editing instruction might be: "I need to go to Jerusalem. Scratch that, Tel Aviv." These have been handled differently by different approaches and models in ASR and speech, and the question is really scope: which of these makes sense?

3.16 The objective (19:27)

What I'd like for my own use: rather than a huge multimodal model capable of processing everything, and therefore very heavy, take a pure ASR model and adapt it so that in a single forward pass it not only does the phonetic rendering but also reasons and produces text that follows these instructions. I don't know if this is architecturally possible: one model that encapsulates two passes, the first pure ASR, the second post-processing. Same result for the user; internally, two different processes going on. I'm not sure, but that's the idea.

3.17 The prototype (20:25)

For the prototype on Hugging Face: take my training dataset, which is basically the kitchen sink, everything I've recorded, stratified by all these types. Isolate only the post-processing tasks, and within those, isolate the ones I've described here as core, ambitious and experimental.

3.18 Whispered sentences (21:03)

There's one in there I've called whispered sentences, and it's an important one for me. Here's the use case. Since I became a voice-first user, sometimes I'm in bed and my wife is saying "stop talking to your phone". I remember asking ChatGPT way back whether Whisper could transcribe whispered speech. Then I just tried it, and it was quite good. I thought, this is actually super useful: maybe you have to write something discreetly, maybe you're trying not to wake the kids, or the wife, or the husband. Why not include an actual small corpus of whispered sentences in the training subset, so

there's explicit training data on whispered speech? That's why I created a whispered dataset, and I'd very much like to have it included for that reason.

3.19 Data and prior art (22:00)

The datasets I have at the moment might need to be beefed up for all of these tasks, paragraph inference and the others. I'd also like a survey of prior art, and parallel art as well: ASR models that support a paragraph break as a token, and anything else. I'd be surprised if there weren't other attempts out there to create a unified ASR post-processing model. (I don't even like that name very much, but it's accurate to the lineage, and that's the objective.)

3.20 Stages (22:47)

There are a few stages. First, setting up the repository with a cleaned-up version of this transcript and a spec from it. Then research on existing attempts, research on feasibility, and research on candidate ASR models that might be better fits than Parakeet, although I'd be reluctant not to use it, because I've become very fond of it.

3.21 Open question: English-only? (23:39)

The other question I have is within the Parakeet models. This would be an English model. If I bake in my code-switching support, I presume it can still be a model that explicitly supports only English? That's an open question: whether it has to be multilingual if code-switching is part of it.

3.22 Correction: domain vocab is out of scope (23:54)

Actually, I'll correct myself and end by clarifying: the idea for this one is to isolate, in the proof of concept at least, the post-processing elements, so that future trainings of a model that integrates niche domain vocab (say my tech model) build on an established base that does the post-processing reliably. So any domain vocab is out of scope for this one. This is the experimental prototype, just for the post-processing tasks combined.

3.23 Corrections

Mishearings fixed against `transcript-raw.txt`:

Raw	Clean	Note
“into a compute out my own fine tunes”	“into a computer: trying out my own fine-tunes”	
“send it to the ladder”	kept, marked [unclear]	possibly “send it off” or “send it to the LLM”
“Parakeet Nematron”	“Parakeet and Nemotron”	both lines are being fine-tuned (Hebrish Shakshuka is the Nemotron one)
“Beto Akhlaomi”	“Bituach Leumi”	
“Whisperer”	“Whisper”	
“ROCM”	“ROCm”	
“evalues”	“evals”	
“tac flow cap”	“tech vocab”	
“the m’s”	“ahs”	
“full starts”	“false starts”	
“adding instruction”	“editing instruction”	
“Tiberius”	“Tiberias”	
“Candidus ASR models”	“candidate ASR models”	
“S1 Mini by Superwhisper”	“S1-mini by Superwhisper”	
“experimentary”	“experimental”	the speaker’s own coinage; same meaning
“Moonshine and Nvidia”	“Moonshine and NVIDIA”	

At 23:54 the speaker revises the scope: domain vocabulary, including code-switching, is out of scope for Proofed. The question at 23:39 is left as spoken; `docs/development-plan.md` records the resulting decision.

EXECUTIVE RESEARCH REPORT

Proofed: phase 1 feasibility

Can a small on-device ASR model be fine-tuned to write clean, proofed text in one pass?

Claude Code for Daniel Rosehill · 24 October 2026

Researched by	Claude Code · Opus 5.5 (claude-opus-5-5), orchestrator and synthesis; five research agents, each Opus 5.5 (claude-opus-5-5)
Started	2026-10-24 14:18:08 +03:00
Completed	2026-10-24 14:43:30 +03:00
Sources	125 source entries, about 115 distinct works (some papers carry more than one id; e.g. Jamshid Lou & Johnson 2020 is S0201, S0202, S0329 and S0401)
Method	Five parallel web-research agents (paragraph token, prior art, base models, feasibility, training datasets), each with its own notes and sources; tokenizer and leaderboard checks run locally; synthesis and spot-checks of load-bearing figures by the orchestrator
Requested by	Daniel Rosehill, 2026-10-24 (“yes please” to running Proofed phase 1 through modular-research:research-report)

Summary

Yes, it is viable as a research prototype. End-to-end ASR models of every family (CTC, attention, transducer) learn to delete disfluencies and to write formatted text when trained on clean targets, and with a pretrained acoustic model the single-model route matched or beat ASR followed by a cleanup model. Parakeet TDT 0.6B v2 remains the best phone-sized English base. Nobody has published a Parakeet or transducer fine-tune for clean output, nor end-to-end handling of dictation self-corrections, same-speaker asides or paragraphs, so those are open and are what Proofed would be first to show. The binding constraint is data: about 42 minutes of in-scope recordings exist, with none for self-corrections; public corpora plus TTS synthesis must supply most of the rest.

1 Bottom line

Proceed to phase 2. The core tier (fillers, repeats, false starts, numbers, currency) is supported by published evidence and needs modest data with replay. The ambitious and experimental tiers (self-corrections, paragraphs, asides) are untested anywhere and should be run as separate, measured experiments, exactly as the plan already intends. Keep Parakeet TDT 0.6B v2 as the base; add Qwen3-ASR-0.6B as a comparison arm for the tasks a transducer may find hard. Use the ^ sentinel for paragraphs now. Budget for synthetic audio (TTS) for self-corrections, asides and the long tail of numbers, and protect against forgetting with replay and, if needed, adapters.

2 Key findings

#	Finding	Confidence
1	End-to-end models learn disfluency removal from clean targets; with pretraining (HuBERT-CTC) they beat the two-stage pipeline, 12.2% vs 13.1% WER [S0203]	High
2	Deletion data needs are modest: ~4.7 h of disfluent data gave 8.7% relative WER reduction and 47 h gave 22.5% in an industrial RNN-T, mixed with ordinary data [S0403]	Medium
3	Written-form numbers train in at scale (SPGISpeech, 5,000 h) but long or rare numbers are the weak spot; Google's best RNN-T numerics used spoken form plus a small denormaliser and 370 h of TTS numerics [S0405, S0406]	Medium
4	Parakeet v2′s tokenizer has no newline; the single-token sentinel ^ (id 918) round-trips exactly and needs no model surgery [S0101]	High
5	Parakeet v2 is still the most accurate open phone-sized English model (Open ASR Leaderboard 6.02, Oct 2026); Qwen3-ASR-0.6B (6.24) is the best alternative and can emit newlines [S0323, S0328]	High
6	Closest prior art is FluentWhisper (June 2026), a Whisper LoRA on ~23k synthetic pairs; self-reported 3.4% vs 9.4% WER on DisfluencySpeech; no Parakeet equivalent found [S0205]	Medium
7	No paper found on end-to-end dictation self-corrections, same-speaker asides or paragraph output; all documented products use ASR followed by a text model or LLM [S0215–S0219]	Medium (absence)
8	Forgetting is severe without care (LibriSpeech test-other 5.12 → 51.08% WER after a narrow full fine-tune) and largely avoided with joint adapters (5.79%) or replay [S0411, S0403]	High
9	Acted (read-aloud) to natural disfluency transfer has never been tested; this is the largest scientific risk [S0407]	High (that it is a gap)
10	Redistributable public audio covers fillers, repeats, false starts and replay; nothing public covers self-corrections or asides, and SPGISpeech's licence excludes it [notes 05]	Medium

3 Findings

3.1 Paragraphs and Parakeet's tokenizer

Parakeet TDT 0.6B v2 uses a 1,024-piece SentencePiece BPE vocabulary with `nmt_nfkc` normalisation, which silently turns `\n` into a space; there is no newline, ¶ or § piece [S0101, S0102]. Three routes exist:

- **Sentinel (recommended now).** ^ is a single token (id 918) that round-trips exactly (¡ 1012 and ¿ 904 also work). Train targets with ^ at breaks and replace it with a blank line after decoding. No architecture change.
- **New token.** NeMo's `change_vocabulary` rebuilds the decoder and joint at random initialisation and, as read in the code (not run), may rebuild the loss without the TDT settings [S0103, S0104]. The

safer variant is weight surgery: insert one row at index 1024, before the blank and the five duration outputs.

- **Different base.** Whisper, Moonshine, Kyutai, Granite 4.0 speech and Qwen3-ASR tokenizers keep newlines; Parakeet v3, Nemotron streaming and Granite 5.0 strip them [S0328]. No open ASR model is documented to emit paragraphs; AssemblyAI and Deepgram segment paragraphs after recognition, on the API side [S0107–S0109].

3.2 Prior art

- **Research, conversational speech only.** Jamshid Lou & Johnson (EMNLP Findings 2020) trained CTC, LSTM and Transformer models on fluent Switchboard targets; all learned removal but were slightly worse than ASR plus a detector (Transformer DER 24.0 vs pipeline 22.2) [S0201]. Bassi et al. (arXiv, Sep 2023) reversed this with pretrained HuBERT-CTC: 12.2% vs 13.1% WER [S0203]. The abstract was checked directly: end-to-end models “match or exceed” two-stage models.
- **FluentWhisper** (HF Space, 2026-06-13): rank-16 LoRA on whisper-large-v3-turbo, ~23k Kokoro-TTS pairs with injected disfluencies; reports 3.4% vs 9.4% WER on DisfluencySpeech (N=250, one acted speaker) and says it loses intentional repeats, digits and some self-repairs [S0205]. Self-reported, hackathon scale.
- **Numbers.** Google (Peyser et al., 2019): written-domain RNN-T was far worse on long numbers than spoken-domain plus a neural denormaliser (50.9 vs 11.3 WER), slightly better on real-audio numeric sets [S0206]. Table read from misaligned HTML.
- **Asides.** Device-directed speech detection is always a separate classifier [S0211–S0214]. Google (ASRU 2023) added primary/non-primary speaker tokens to an RNN-T so dictation skips *other* speakers [S0210]; Proofed’s case, the *same* speaker turning aside, is untouched. Addressee shifts are detectable from prosody (EER 12.6% vs 23.1% lexical) [S0213].
- **Products.** Superwhisper S1 (S1-mini is a Qwen3-0.6B text model), Wispr Flow (fine-tuned Llama after ASR) and AssemblyAI (separate LLM pass for self-corrections) are all two-stage [S0215–S0219]. Apple, Google Eloquent, Aqua and Willow do not disclose architecture.

3.3 Candidate base models

Model	Params	Decoder	Newline	Phone runtime	OAL (Oct 2026)
parakeet-tdt-0.6b-v2 (keep)	600M	TDT transducer	no	sherpa-onnx	6.02
Qwen3-ASR-0.6B	938M	LLM	yes	sherpa-onnx (Android)	6.24
canary-180m-flash	182M	attention (4 layers)	untested	sherpa-onnx	not listed
nemotron-speech-streaming-en-0.6b	618M	cache-aware RNN-T	no	sherpa-onnx	6.60
parakeet-tdt-0.6b-v3	627M	TDT	no	sherpa-onnx	6.18

Sources: [S0301–S0331]. Parakeet v2 also matches Daniel’s field experience that v2 beats v3 for English dictation [S0331]. The shortlist, in order: Qwen3-ASR-0.6B as the LLM-decoder control

arm (newlines, Apache-2.0, official fine-tuning recipe); canary-180m-flash as a cheap transducer-vs-attention test inside NeMo; Nemotron streaming as the streaming option, expected weakest on self-corrections. parakeet-unified-en-0.6b scores 5.91 but cannot be fine-tuned yet.

Inference, not sourced: Proofed's core edits are monotonic deletions and in-order rewrites, which a transducer handles by emitting blanks. Edits that need later context ("Tuesday, no, Wednesday") or reorder ("fifty dollars" → "$50") are harder; offline TDT's full-context encoder should carry that context, cache-aware streaming models cannot.

3.4 Feasibility evidence

- **Deletion is learned by every family:** CTC, attention and RNN-T [S0401–S0404]. In the Amazon RNN-T, the baseline's failure mode was insertions, and ordinary-speech WER did not move [S0403].
- **Formatting:** Conformer-CTC on SPGISpeech reached 6.0% vs 5.7% for an attention model, with half to two-thirds of the error from formatting [S0405].
- **Whispered speech:** learnable from about 11–26 h; WavLM fine-tuned reached 9.22% whispered WER. Stock Whisper large-v2 loses only ~2 points on whispers (16.69% vs 18.80%), so Parakeet's whisper gap should be measured before deciding how much to record [S0409, S0410].
- **Forgetting:** NVIDIA's transducer paper: full fine-tune 5.12 → 51.08% test-other; constrained fine-tune 29.78%; encoder adapters 6.24%; joint adapters 5.79% (checked against the paper's table) [S0411]. Replay with 0.6% of the original data closed over 40% of the gap [S0412].
- **Acted vs natural:** no study tests transfer from acted or synthetic disfluencies to natural ones [S0407, S0414]. Daniel's planned `Parakeet-Dictation-Cleanup-Eval` (40 spontaneous dictations, still a placeholder) would measure exactly this [S0415].

3.5 Training data map

Daniel's own recordings (counted from the Hub metadata, 2026-10-24; train split; all CC BY 4.0, one speaker): disfluency 51 takes (12.4 min), repetitions 18 (3.6), false starts 3 (0.3), numerics ~113 (~13, before removing code-switching rows), currency 116 (11.1), self-corrections 0, asides 2 (0.3), paragraphs 3 (0.7), whispered 3 (0.4), plus 345 plain-English takes (34.4 min) for replay. The recorder app's new Proofed mode (train.hebrish.app, release 25) serves the 100 self-correction, 100 aside, 140 whispered and 49 paragraph prompts that already exist.

Public datasets (licences read by agent 05; full table in `notes/05-datasets.md`):

Use	Datasets
Redistributable, in a public blend	DisfluencySpeech (Apache-2.0, ~10 h acted, clean targets ready); AMI and ICSI (CC BY 4.0, ~100 h and ~70 h natural meetings; clean targets must be derived); replay from LibriSpeech-PC (CC BY 4.0), Libriheavy (Apache-2.0), VoxPopuli-en (CC0), People's Speech (CC-BY/SA), Granary-en (CC BY 4.0)
Training time only (grey for a published model)	Switchboard/Fisher (LDC), GigaSpeech (non-commercial), EARS and Expresso whisper subsets (CC BY-NC), wTIMIT (no licence), Common Voice (re-hosting barred), Earnings-21/22 (keep as a natural numbers test set)
Excluded	SPGISpeech (academic and internal use only), FluentWhisper's data (no licence), LibriStutter / LLM-Dys / Libri-Dys (no licence; clinical stuttering), SEP-28k (audio not distributed)
Synthesis inputs	Disfl-QA text (CC BY 4.0, ~12k disfluent questions, >90% corrections or restarts) for self-corrections; NeMo-text-processing (Apache-2.0) to

Use	Datasets
	verbalise digit targets for TTS; Kokoro-82M voices (Apache-2.0). LARD is CC BY-NC-SA: reimplement its rules rather than ship its output

4 Implications for this project

1. **Phase 2 data audit** should start from the counts above. The gaps are self-corrections (none), asides, paragraphs, whispered and false starts (single digits).
2. **Recording:** use the recorder's Proofed mode, which gives each subtask an equal share of cards, so the empty tasks fill first.
3. **Synthesis:** plan a TTS pipeline (Kokoro voices, or Daniel's own voice clone for a single-speaker match) driven by Disfl-QA patterns for self-corrections, scripted asides, and NeMo TN for numbers and currency.
4. **Merged dataset:** a public, ungated `danielrosehill/Proofed-ASR-Cleanup-Training-Data` is being built from Daniel's in-scope takes (no code-switching rows), keeping the source train/test split; public corpora are referenced by manifest and pulled at training time.
5. **Paragraphs:** adopt the ^ sentinel; revisit token surgery only if the sentinel fails.
6. **Forgetting:** start with replay (~10% LibriSpeech-PC, per `ASR-Dataset-Blending`) and a low learning rate; keep joint adapters as the fallback.
7. **Evaluation:** record the natural-dictation eval set early; it is the only way to test acted-to-natural transfer. Consider an ablation that emits a removable tag instead of deleting, since deletion is irreversible [S0403, S0404].

5 Open questions

Needs Daniel:

1. **Licensing policy for training data:** may a published Proofed model be trained on non-commercial or LDC data (Switchboard, GigaSpeech, EARS), or only on permissively licensed data? No source settles whether model weights inherit these terms.
2. **Recording budget** per thin category, now that the counts are known (plan question 5).
3. **Paragraph route:** confirm the ^ sentinel (plan question 2).
4. **Tiers:** confirm the tier table, especially the "open" row (plan question 3).
5. **Synthetic voice:** Kokoro's many voices, or Daniel's own voice clone for synthetic takes?

Open research questions: acted-to-natural transfer; transducer vs attention decoder on self-corrections (no study exists); Parakeet's whispered-speech gap; whether stacking several tasks in one fine-tune causes interference (not studied).

6 Sources

Grouped by sub-question, as the agents recorded them. Confirmed and inferred status is given in each note under `notes/`.

6.1 01 paragraph token

- [S0101] Local inspection of parakeet-tdt-0.6b-v2 SentencePiece tokenizer — local check, 2026-10-24. Tokenizer streamed out of https://huggingface.co/nvidia/parakeet-tdt-0.6b-v2/resolve/main/parakeet-tdt-0.6b-v2.nemo (repo sha ae9ad070, lastModified 2026-06-29) to /var/tmp/claude-1000/-home-daniel-repos-github-ai-models-Proofed/8a4b2063-9fd8-4b45-a5d6-83706635a298/scratchpad/research/01-dl/nemo/705f11d22dc04b169effc35ce5cd1361_tokenizer.model (sha256 5a3a82c4…b0a0) plus model_config.yaml; inspected with `uv run --no-project --with sentencepiece --with protobuf python -I .../scratchpad/research/01-scripts/inspect_sp.py <model>` and `sentinel.py`. Accessed 2026-10-24. Supports: vocab 1024 BPE, no newline/¶/§ pieces, nmt_nfkc maps to space, ^/¡/¿ single-token round-trip, config (EncDecRNNTBPEModel, TDT, 5 extra outputs).
- [S0102] tokens.txt, sherpa-onnx-nemo-parakeet-tdt-0.6b-v2-int8 — csukuangfj (k2-fsa), undated. https://huggingface.co/csukuangfj/sherpa-onnx-nemo-parakeet-tdt-0.6b-v2-int8/resolve/main/tokens.txt. Accessed 2026-10-24. Supports: 1025 entries (1024), identical pieces to .nemo tokenizer.vocab (diff check).
- [S0103] nemo/collections/asr/models/rnnt_bpe_models.py and rnnt_models.py — NVIDIA NeMo, main branch as fetched 2026-10-24. https://github.com/NVIDIA/NeMo/blob/main/nemo/collections/asr/models/rnnt_bpe_models.py. Accessed 2026-10-24. Supports: change_vocabulary deletes and re-instantiates joint and decoder (encoder kept); rebuilds RNNTLoss without loss_name/tdt kwargs, unlike **init**.
- [S0104] nemo/collections/asr/modules/rnnt.py — NVIDIA NeMo, main branch as fetched 2026-10-24. https://github.com/NVIDIA/NeMo/blob/main/nemo/collections/asr/modules/rnnt.py. Accessed 2026-10-24. Supports: joint output = num_classes + 1 (blank) + num_extra_outputs; decoder blank_idx = vocab_size; Embedding(vocab_size+1, padding_idx=blank).
- [S0105] Local inspection of Whisper large-v3 tokenizer.json — local check, 2026-10-24. ~/.cache/huggingface/hub/models–Systran–faster-whisper-large-v3/snapshots/edaa852ec7e145841d8ffdb056a99866b5f0a478/tokenizer.json, `python3 -I` lookup of model.vocab. Accessed 2026-10-24. Supports: ‘Ċ’ (newline) id 198, ‘¶’ id 114 present.
- [S0106] Tutorial: How to streamline your writing process with Whisper and GPT-4 — Ignorance.ai, undated. https://www.ignorance.ai/p/tutorial-how-to-streamline-your-writing. Accessed 2026-10-24 (search result summary only, not read in full). Supports (weak): raw Whisper output has no paragraph breaks.
- [S0107] Get paragraphs in transcript (API reference) — AssemblyAI, undated. https://www.assemblyai.com/docs/api-reference/transcripts/get-paragraphs (and …/get-sentences). Accessed 2026-10-24. Supports: GET /v2/transcript/{id}/paragraphs segments an existing transcript semantically; API-side.
- [S0108] Paragraphs — Deepgram docs, undated. https://developers.deepgram.com/docs/paragraphs. Accessed 2026-10-24. Supports: paragraphs=true, breaks based on punctuation and speaker/channel changes; enables punctuation.
- [S0109] Smart Format — Deepgram docs, undated. https://developers.deepgram.com/docs/smart-format. Accessed 2026-10-24. Supports: smart_format applies Punctuation and Paragraphs at minimum for English.
- [S0110] nvidia/parakeet-tdt-0.6b-v2 model card — NVIDIA / Hugging Face, lastModified 2026-06-29. https://huggingface.co/nvidia/parakeet-tdt-0.6b-v2. Accessed 2026-10-24. Supports: PnC output, Granary ~120k h training data; no mention of newlines/paragraphs or vocab size.

6.2 04 feasibility

- [S0401] End-to-End Speech Recognition and Disfluency Removal (Jamshid Lou & Johnson) — Findings of EMNLP 2020 / arXiv:2009.10298, 2020. https://ar5iv.labs.arxiv.org/html/2009.10298. Accessed 2026-10-24. Supports: CTC, LSTM-AED and Transformer all learn fluent output from SWBD (260 h) + Fisher (~2,000 h, silver); E2E slightly worse than pipeline (test DER 26.2/25.6/24.0 vs 23.3/22.6/22.2); corrections DER CTC 33.5, Seq2Seq 29.5, Transformer 25.8; repetitions sometimes kept.
- [S0402] End-to-End Speech Recognition and Disfluency Removal with Acoustic Language Model Pretraining — arXiv:2309.04516, 2023. https://ar5iv.labs.arxiv.org/html/2309.04516. Accessed 2026-10-24. Supports: HuBERT Conformer-CTC fluent E2E 12.2% WER vs two-stage 13.1% on Switchboard; wav2vec2 init much worse.
- [S0403] Improved Robustness to Disfluencies in RNN-Transducer Based Speech Recognition (Mendelev et al., Amazon) — ICASSP 2021 / arXiv:2012.06259, 2020-12-11. https://arxiv.org/abs/2012.06259 (PDF read locally via pdftotext). Accessed 2026-10-24. Supports: RNN-T, ~2,300 h ordinary + 47 h disfluent; relative WERR by disfluent fraction 1/10 8.7%, 1/4 13.9%, 1/2 18.3%, full 22.5% (disfl test), stutter 16.4%; no ordinary-test regression; partial-word tag beats deletion; stutter set = 11 speakers reading prompts.
- [S0404] Streaming Joint Speech Recognition and Disfluency Detection (Futami et al., Sony/CMU) — ICASSP 2023 / arXiv:2211.08726, 2022-11. https://ar5iv.labs.arxiv.org/html/2211.08726. Accessed 2026-10-24. Supports: streaming Transformer AED; E2E fluent-only DR-WER 21.7 vs multi-task 21.3 vs BERT pipeline 22.6 on SWBD; argument that fluent-only output loses verbatim text and cannot undo over-deletion.
- [S0405] SPGISpeech: 5,000 hours of transcribed financial audio for fully formatted end-to-end speech recognition (O'Neill et al.) — Interspeech 2021 / arXiv:2104.02014, 2021-04. https://ar5iv.labs.arxiv.org/html/2104.02014. Accessed 2026-10-24. Supports: non-verbatim (fillers/false starts removed), denormalized numbers; Conformer AED 5.7% vs Conformer-CTC 6.0% orthographic WER; currency slices excluded.
- [S0406] Improving Performance of End-to-End ASR on Numeric Sequences (Peyser, Zhang, Sainath, Wu, Google) — Interspeech 2019 / arXiv:1907.01372, 2019-07. https://arxiv.org/abs/1907.01372 (PDF read locally). Accessed 2026-10-24. Supports: RNN-T written-domain numerics; 370 h TTS at 10% of batches on 43M-utterance base; 65% relative WER gain on short numbers; best = spoken-domain + neural denormer; up to 8x WER reduction.
- [S0407] DisfluencySpeech – Single-Speaker Conversational Speech Dataset with Paralanguage (Wang & Herremans) — TENCON 2024 / arXiv:2406.08820, 2024-06-13. https://arxiv.org/abs/2406.08820 (PDF read locally). Accessed 2026-10-24. Supports: ~10 h, one speaker acting Switchboard disfluencies, three transcript tiers; authors flag acting vs spontaneity as a limitation; TTS only, no ASR transfer test.
- [S0408] Thutmose Tagger: Single-pass neural model for Inverse Text Normalization (NVIDIA) — Interspeech 2022 / arXiv:2208.00064, 2022-07. https://arxiv.org/abs/2208.00064 (PDF read locally). Accessed 2026-10-24. Supports: ITN tagging assumes monotonic alignment; non-monotonic pairs discarded except common reorderings ("ten thousand dollars" → "$10,000") encoded by swap tags.
- [S0409] Leveraging Self-Supervised Models for Automatic Whispered Speech Recognition — arXiv:2407.21211, 2024-07. https://arxiv.org/abs/2407.21211 (PDF read locally). Accessed 2026-10-24. Supports: Whisper large-v2 16.69% normal vs 18.80% whispered WER; WavLM fine-tuned on 17 h wTIMIT (11 h whispered, 20 spk) + 3 h CHAINS normal (36 spk) → 9.22% whispered WER.

- [S0410] End-to-end Whispered Speech Recognition with Frequency-weighted Approaches and Pseudo Whisper Pre-training (Chang et al.) — SLT 2021 / arXiv:2005.01972, 2020-05. https://arxiv.org/abs/2005.01972 (PDF read locally). Accessed 2026-10-24. Supports: wTIMIT ~26 h whispered + ~26 h normal, 48 speakers, 450 sentences, sentence overlap in original split; E2E whispered PER 59.5 vs normal 29.7, GEO SpecAug 48.3; 19.8% PER / 44.4% CER relative reduction; small whispered set suffices after pretraining.
- [S0411] Damage Control During Domain Adaptation for Transducer Based Automatic Speech Recognition (Majumdar et al., NVIDIA) — SLT 2022 / arXiv:2210.03255, 2022-10-06. https://arxiv.org/abs/2210.03255 (PDF read locally). Accessed 2026-10-24. Supports: NeMo transducer; full FT on Speech Commands sends LS test-other 5.12 → 51.08 (constrained FT 29.78); encoder adapters 6.24, joint adapters 5.79; dialect FT 5.11 → 7.68 vs joint adapter 5.40.
- [S0412] Using Adapters to Overcome Catastrophic Forgetting in End-to-End ASR / Continual learning for monolingual E2E ASR (Vander Eeckt & Van hamme) — ICASSP 2023 / arXiv:2203.16082; arXiv:2112.09427, 2022. https://arxiv.org/abs/2203.16082. Accessed 2026-10-24 (2112.09427 claim from search summary only, not read in full). Supports: task-specific adapters + regularised shared params beat baselines without stored data; best CL method closes >40% of FT-vs-joint gap with 0.6% of original data.
- [S0413] nvidia/parakeet-tdt-0.6b-v2 model card — NVIDIA / Hugging Face, undated (fetched 2026-10-24). https://huggingface.co/nvidia/parakeet-tdt-0.6b-v2. Accessed 2026-10-24. Supports: ~120k h (10k human, 110k pseudo-labelled YTC/YODAS/Librilight), PnC preserved, “robust performance on spoken numbers”; no verbatim/disfluency or ITN statement; SPGISpeech eval only.
- [S0414] Spontaneous Speech Effects In Large Vocabulary Speech Recognition Applications (Butzberger, Murveit, Shriberg, Price, SRI) — DARPA Speech and Natural Language Workshop 1992 (ACL Anthology H92-1068). https://aclanthology.org/H92-1068.pdf. Accessed 2026-10-24. Supports: read 33% vs spontaneous 43% WER; disfluent spontaneous 56% vs fluent spontaneous 32%; gap due to disfluencies.
- [S0415] Local check: danielrosehill/Parakeet-Dictation-Cleanup-Pairs and -Eval — ~/.cache/huggingface/hub/datasets–danielrosehill–Parakeet-Dictation-Cleanup-Pairs/snapshots/cae5e1cb258c44156b253c260a6490d4fdc079dc/README.md, ~/.cache/huggingface/hub/datasets–danielrosehill–Parakeet-Dictation-Cleanup-Eval/snapshots/9cda6a2403c40cbca144ef3bc2d3919ffe207090/README.md; plus https://huggingface.co/api/datasets/danielrosehill/Parakeet-Dictation-Cleanup-Eval (files .gitattributes, README.md; lastModified 2026-10-16) and …/Parakeet-Dictation-Cleanup-Pairs (HTTP 401 unauthenticated). Accessed 2026-10-24. Supports: both are placeholders (status 2026-10-16), text-to-text for the S1-mini LoRA; Pairs planned ~400 synthetic + 50 no-op pairs; Eval planned 40 spontaneous single-speaker dictations, hand-cleaned.

6.3 02 prior art

- [S0201] End-to-End Speech Recognition and Disfluency Removal (arXiv 2009.10298, ar5iv HTML tables 3–4) — Jamshid Lou & Johnson, arXiv, 2020-09-22. https://ar5iv.labs.arxiv.org/html/2009.10298. Accessed 2026-10-24. Supports: E2E models trained on fluent Switchboard targets learn fluent output but are slightly worse than ASR+disfluency-detector pipeline (e.g. Transformer test DER 24.0 vs 22.2).
- [S0202] End-to-End Speech Recognition and Disfluency Removal — ACL Anthology (Findings of EMNLP 2020), 2020-11. https://aclanthology.org/2020.findings-emnlp.186/. Accessed 2026-10-24. Supports: venue, date, pages 2051–2061.
- [S0203] End-to-End Speech Recognition and Disfluency Removal with Acoustic Language Model Pretraining — Bassi, Duregon, Jalagam, Roth, arXiv 2309.04516, 2023-09-08. https://arxiv.org/abs/

2309.04516 (numbers from https://ar5iv.labs.arxiv.org/html/2309.04516). Accessed 2026-10-24. Supports: HuBERT-CTC E2E 12.2% WER vs two-stage 13.1% on Switchboard fluent refs; pretraining objective matters; no FER/DER reported.

- [S0204] End-to-End Spoken Grammatical Error Correction — Qian, Ma, Bannò, Gales, Knill, arXiv 2506.18532, 2025-06-23 (v2 2026-10-13; Speech Communication). https://arxiv.org/abs/2506.18532. Accessed 2026-10-24. Supports: cascaded vs E2E Whisper comparison for fluent/GEC output; pseudo-labelling 77→2,500+ h (abstract only).
- [S0205] FluentWhisper Space README — build-small-hackathon / Hugging Face, created 2026-06-13, last modified 2026-06-14. https://huggingface.co/spaces/build-small-hackathon/fluentwhisper (README via /raw/main/README.md; Space paused). Accessed 2026-10-24. Supports: LoRA on whisper-large-v3-turbo, ~23k synthetic Kokoro-TTS pairs from LibriSpeech text, 3.4% vs 9.4% WER on DisfluencySpeech (N=250), aggressive deletion loses repeats/digits/some self-repairs.
- [S0206] Improving Performance of End-to-End ASR on Numeric Sequences — Peyser, Zhang, Sainath, Wu (Google), arXiv 1907.01372 / Interspeech 2019, 2019-07-01. https://ar5iv.labs.arxiv.org/html/1907.01372. Accessed 2026-10-24. Supports: written-domain RNN-T weak on long numerics (W0 92.7, W2 50.9 vs spoken+denormer 11.3 on TAIL_LONG); written slightly better on real-audio numeric sets.
- [S0207] End-to-end Joint Punctuated and Normalized ASR with a Limited Amount of Punctuated Training Data — Cui, Sheikh, Sadeghi, Vincent, arXiv 2311.17741, 2023-11-29 (EUSIPCO 2025). https://arxiv.org/abs/2311.17741. Accessed 2026-10-24. Supports: single conditioned decoder emits punctuated or normalised output; 42% rel. PC-WER reduction vs Whisper-base; works with 5% punctuated data.
- [S0208] Four-in-One: A Joint Approach to ITN, Punctuation, Capitalization, and Disfluency for ASR — Tan, Behre, Kibre, Alphonso, Chang, arXiv 2210.15063, 2022-10-26. https://arxiv.org/abs/2210.15063. Accessed 2026-10-24. Supports: post-ASR transformer tagger + WFST; separate text-side stage.
- [S0209] Universal-2-TF: Robust All-Neural Text Formatting for ASR — Khare, Peyash, Vanzo, Yoshioka, arXiv 2501.05948, 2025-01-10. https://arxiv.org/abs/2501.05948. Accessed 2026-10-24. Supports: two-stage neural text formatter (token classifier + seq2seq) for punctuation, truecasing, ITN.
- [S0210] Improved Long-Form Speech Recognition by Jointly Modeling the Primary and Non-primary Speakers — Arumugam, Chang, Sainath, Prabhavalkar, Wang, Bijwadia (Google), arXiv 2312.11123 / ASRU 2023, 2023-12-18. https://ar5iv.labs.arxiv.org/html/2312.11123. Accessed 2026-10-24. Supports: primary/non-primary speaker tokens in ASR vocab; dictation domain should not transcribe non-primary speakers; dictation WER 3.6 unchanged; 55% fewer long-form deletions.
- [S0211] Device-directed Utterance Detection — Mallidi, Maas, Goehner, Rastrow, Matsoukas, Hoffmeister (Amazon), arXiv 1808.02504 / Interspeech 2018, 2018-08-07. https://arxiv.org/abs/1808.02504. Accessed 2026-10-24. Supports: separate classifier on acoustic + 1-best + decoder features, 5.2% EER combined.
- [S0212] Multimodal Data and Resource Efficient Device-Directed Speech Detection with Large Foundation Models — Wagner et al., arXiv 2312.03632, 2023-12-06. https://arxiv.org/abs/2312.03632. Accessed 2026-10-24. Supports: DDSD as separate frozen-LLM classifier over ASR 1-best, decoder signals and audio encoder; ≤80k training examples.
- [S0213] Learning When to Listen: Detecting System-Addressed Speech in Human-Human-Computer Dialog — Shriberg, Stolcke, Hakkani-Tür, Heck, Interspeech 2012. https://www.isca-archive.org/interspeech_2012/shriberg12_interspeech.html. Accessed 2026-10-24. Supports: users change speaking style by addressee; prosodic EER 12.6% vs lexical 23.1%, combined 11.1%.

- [S0214] Selective Attention System (SAS): Device-Addressed Speech Detection for Real-Time On-Device Voice AI — Kim, Anjum, Banerjee, Abbasi, arXiv 2604.08412, 2026-04-09. https://arxiv.org/abs/2604.08412. Accessed 2026-10-24. Supports: on-device pre-ASR gate, audio-only F1 0.86 on proprietary set.
- [S0215] Introducing the S1 family of models — Superwhisper, 2026-08-19. https://superwhisper.com/blog/s1. Accessed 2026-10-24. Supports: S1-Voice ASR + S1-Language/S1-mini text cleanup pipeline; S1-mini handles fillers, stutters, self-corrections, numbers/dates/currency.
- [S0216] superwhisper/S1-mini model card — Superwhisper / Hugging Face, undated (citation 2026). https://huggingface.co/superwhisper/S1-mini. Accessed 2026-10-24. Supports: Qwen3-0.6B fine-tune, text-in/text-out, Apache 2.0, 94.8% token accuracy on 7,519 held-out cases.
- [S0217] Technical challenges behind Flow — Sahaj Garg, Wispr Flow, 2025-09-26. https://wisprflow.ai/research/technical-challenges. Accessed 2026-10-24. Supports: separate ASR (<200 ms) and LLM (<200 ms) inference within 700 ms budget.
- [S0218] Wispr Flow customer story — Baseten, undated (© 2026). https://www.baseten.co/resources/customers/wispr-flow/. Accessed 2026-10-24. Supports: Flow's cleanup step is a fine-tuned Llama after speech recognition; <700 ms p99.
- [S0219] Dictation cleanup — AssemblyAI blog, 2026-09-02. https://assemblyai.com/blog/dictation-cleanup. Accessed 2026-10-24. Supports: fillers removed by default (disfluencies:true keeps them); self-corrections handled in a separate LLM rewrite pass.
- [S0220] Google AI Edge Eloquent AI dictation app — Android Authority, 2026-04-07. https://www.androidauthority.com/google-ai-edge-eloquent-ai-dictation-app-3655274/. Accessed 2026-10-24. Supports: removes fillers and corrects mid-sentence errors on device, optional Gemini cloud cleanup; models not named.
- [S0221] Apple introduces systemwide dictation — TechCrunch, 2026-06-08. https://techcrunch.com/2026/06/08/apple-introduces-systemwide-dictation. Accessed 2026-10-24. Supports: Gemini-based Apple Intelligence model corrects spelling/punctuation/capitalisation; filler/self-correction handling and architecture not stated.
- [S0222] How to use the Avalon ASR model — Aqua Voice, undated (© 2026). https://aquavoice.com/guide/how-to-use-avalon. Accessed 2026-10-24. Supports: Avalon is Aqua's own ASR model; no statement on cleanup or architecture.
- [S0223] abhiram3040/simplewords-dictation-cleanup-v2 model card — Hugging Face, undated. https://huggingface.co/abhiram3040/simplewords-dictation-cleanup-v2. Accessed 2026-10-24. Supports: post-ASR Qwen3.5-2B QLoRA cleanup; v1 "self-correction inversion" (kept replaced side), fixed in v2 with self-correction examples.
- [S0224] Evaluating OpenAI's Whisper on Community Conversations — Cortico, 2023-06-14. https://cortico.ai/news/insights/evaluating-openai-s-whisper-on-community-conversations/. Accessed 2026-10-24. Supports: Whisper removes ums/uhs but transcribes most other disfluencies.
- [S0225] Kid-Whisper: Towards Bridging the Performance Gap in ASR for Children vs. Adults — Attia et al., arXiv 2309.07927 (v3 2024-05-15). https://arxiv.org/pdf/2309.07927. Accessed 2026-10-24. Supports: "Whisper is known to perform implicit disfluency removal as a result of its data normalization (Radford et al. 2023)".
- [S0226] DisfluencySpeech – Single-Speaker Conversational Speech Dataset with Paralanguage — Wang & Herremans, arXiv 2406.08820, 2024-06-13. https://arxiv.org/abs/2406.08820. Accessed 2026-10-24. Supports: ~10 h single speaker, three transcripts removing non-speech events, non-sentence elements, false starts.

- [S0227] Willow dictation features — Willow Voice, undated. https://willowvoice.com/features/dictation. Accessed 2026-10-24. Supports: removes fillers, fixes grammar, offline on-device mode; no model/architecture stated.

6.4 03 base models

- [S0301] nvidia/parakeet-tdt-0.6b-v2 model card — NVIDIA / Hugging Face, 2025-05-01. https://huggingface.co/nvidia/parakeet-tdt-0.6b-v2. Accessed 2026-10-24. Supports: 600M params, FastConformer-TDT, PnC, CC-BY-4.0, average WER 6.05 on the 8 Open ASR datasets.
- [S0302] nvidia/parakeet-tdt-0.6b-v3 model card — NVIDIA / Hugging Face, 2025-08-14. https://huggingface.co/nvidia/parakeet-tdt-0.6b-v3. Accessed 2026-10-24. Supports: 25 languages, 8,192-token SentencePiece vocab, CC-BY-4.0, average WER 6.34, chunked streaming script.
- [S0303] nvidia/parakeet-tdt_ctc-110m model card — NVIDIA / Hugging Face, undated. https://huggingface.co/nvidia/parakeet-tdt_ctc-110m. Accessed 2026-10-24. Supports: ~114M params, hybrid TDT-CTC, PnC, CC-BY-4.0, mean WER 7.49.
- [S0304] nvidia/nemotron-speech-streaming-en-0.6b model card — NVIDIA / Hugging Face, 2026-03-13 (updated checkpoint). https://huggingface.co/nvidia/nemotron-speech-streaming-en-0.6b. Accessed 2026-10-24. Supports: cache-aware RNN-T, chunks of 80–1120 ms, WER 6.93–8.43 by chunk, NVIDIA Open Model License, GGUF.
- [S0305] nvidia/nemotron-3.5-asr-streaming-0.6b model card — NVIDIA / Hugging Face, 2026-06-04. https://huggingface.co/nvidia/nemotron-3.5-asr-streaming-0.6b. Accessed 2026-10-24. Supports: 40 locales, OpenMDW-1.1, English FLEURS WER 7.91–9.43.
- [S0306] nvidia/parakeet-unified-en-0.6b model card — NVIDIA / Hugging Face, 2026-04-07. https://huggingface.co/nvidia/parakeet-unified-en-0.6b. Accessed 2026-10-24. Supports: unified offline/streaming RNN-T, 5.91 offline average WER, inference only (no training pipeline yet), NVIDIA Open Model License.
- [S0307] nvidia/canary-1b-flash model card — NVIDIA / Hugging Face, undated. https://huggingface.co/nvidia/canary-1b-flash. Accessed 2026-10-24. Supports: 883M AED (32 enc / 4 dec), optional PnC, CC-BY-4.0, mean WER 6.35, NeMo fine-tuning.
- [S0308] nvidia/canary-180m-flash model card — NVIDIA / Hugging Face, undated. https://huggingface.co/nvidia/canary-180m-flash. Accessed 2026-10-24. Supports: 182M AED (17 enc / 4 dec), optional PnC, CC-BY-4.0, mean WER 7.12.
- [S0309] Training and Inference Efficiency of Encoder-Decoder Speech Models (Canary-1B-Flash paper) — Żelasko et al. / arXiv 2503.05931, 2025-03-07 (v2 2025-03-19). https://arxiv.org/abs/2503.05931. Accessed 2026-10-24. Supports: decoder cut 24→4 layers, encoder 32 layers; a smaller decoder may be less resilient to inaccurate labels.
- [S0310] Moonshine: Speech Recognition for Live Transcription and Voice Commands — Jeffries et al. / arXiv 2410.15608v2, 2024-10-22. https://arxiv.org/abs/2410.15608. Accessed 2026-10-24. Supports: tiny 27.1M / base 61.5M, average WER 12.66 / 10.07, 32,768-token byte-level BPE.
- [S0311] moonshine-ai/moonshine-streaming-medium model card — Moonshine AI / Hugging Face, 2026-01-06 (repo created). https://huggingface.co/moonshine-ai/moonshine-streaming-medium. Accessed 2026-10-24. Supports: sliding-window streaming AED, English only, MIT, 80 ms lookahead, hallucination/repetition warning on short or noisy audio.
- [S0312] Moonshine v2: Ergodic Streaming Encoder ASR for Latency-Critical Speech Applications — Kudlur, King, Wang, Warden / arXiv 2602.12241, 2026-02-12. https://arxiv.org/abs/2602.12241. Accessed 2026-10-24. Supports: tiny/small/medium 33.6M/123M/245M, average WER 12.01/7.84/6.65, same tokenizer as v1, M3 latency table.

- [S0313] openai/whisper-large-v3-turbo model card — OpenAI / Hugging Face, undated (repo 2024-10-01). https://huggingface.co/openai/whisper-large-v3-turbo. Accessed 2026-10-24. Supports: 809M, 4 decoder layers (down from 32), MIT, HF fine-tuning route.
- [S0314] distil-whisper/distil-small.en model card — Hugging Face, undated. https://huggingface.co/distil-whisper/distil-small.en. Accessed 2026-10-24. Supports: 166M, short-form WER 12.1, MIT, whisper.cpp/Transformers.js.
- [S0315] kyutai/stt-1b-en_fr model card — Kyutai / Hugging Face, undated (repo 2025-06-17). https://huggingface.co/kyutai/stt-1b-en_fr. Accessed 2026-10-24. Supports: ~1B decoder-only streaming STT, 0.5 s delay, PnC, CC-BY-4.0, no WER on card.
- [S0316] ibm-granite/granite-speech-5.0-470m-turboctc model card — IBM / Hugging Face, 2026-08-25 (multilingual update 2026-10-21). https://huggingface.co/ibm-granite/granite-speech-5.0-470m-turboctc. Accessed 2026-10-24. Supports: 470M Conformer CTC, PnC, 16,384 BPE, Apache-2.0, card's own OAL average 5.51, Colab fine-tuning, mlx-audio.
- [S0317] ibm-granite/granite-4.0-1b-speech model card — IBM / Hugging Face, 2026-03-06. https://huggingface.co/ibm-granite/granite-4.0-1b-speech. Accessed 2026-10-24. Supports: Conformer + Q-former + granite-4.0-1b LLM, Apache-2.0, OAL average 5.52, fine-tuning notebook.
- [S0318] Qwen/Qwen3-ASR-0.6B model card — Qwen / Hugging Face, undated (repo 2026-01-28). https://huggingface.co/Qwen/Qwen3-ASR-0.6B. Accessed 2026-10-24. Supports: Apache-2.0, streaming via vLLM only, English WER (LS 2.11/4.55 etc.).
- [S0319] QwenLM/Qwen3-ASR GitHub README — Qwen, 2026-01-29 release. https://github.com/QwenLM/Qwen3-ASR. Accessed 2026-10-24. Supports: official finetuning directory; 0.6B and 1.7B sizes; Transformers support added 2026-06-26.
- [S0320] mistralai/Voxtral-Mini-4B-Realtime-2602 model card — Mistral AI / Hugging Face, 2026-02. https://huggingface.co/mistralai/Voxtral-Mini-4B-Realtime-2602. Accessed 2026-10-24. Supports: ~4B (3.4B LM + 970M encoder), streaming at 480 ms, Apache-2.0, ExecuTorch/MLX/voxtral.c.
- [S0321] FermionResearch/Phonon-2 model card — Fermion Research / Hugging Face, 2026-09-28 (repo created). https://huggingface.co/FermionResearch/Phonon-2. Accessed 2026-10-24. Supports: compressed derivative of parakeet-tdt-0.6b-v3, 164 MB, CC-BY-4.0, vendor average WER 5.21, Android arm64 CPU libraries.
- [S0322] Edge0/ARK-ASR-0.6B model card — Edge0 / Hugging Face, 2026-05 (repo 2026-05-25). https://huggingface.co/Edge0/ARK-ASR-0.6B. Accessed 2026-10-24. Supports: Whisper-style encoder + Qwen2 0.6B decoder, Apache-2.0, Transformers only, English average 6.55.
- [S0323] Open ASR Leaderboard (English short-form, version "21-10-2026") — Hugging Face hf-audio, last updated 2026-10-22. https://huggingface.co/spaces/hf-audio/open_asr_leaderboard (read from the space's Gradio /config at https://hf-audio-open-asr-leaderboard.hf.space/config). Accessed 2026-10-24. Supports: current average WERs (v2 6.02, v3 6.18, Qwen3-ASR-0.6B 6.24, granite-5.0-470m 6.38, nemotron streaming 6.60, etc.), cleaned and private dataset set, removal of moonshine-base on 2026-07-31.
- [S0324] k2-fsa/sherpa-onnx GitHub release "asr-models" (asset list) — k2-fsa, assets 2024-07 to 2026-07. https://github.com/k2-fsa/sherpa-onnx/releases/tag/asr-models. Accessed 2026-10-24. Supports: which models have sherpa-onnx builds (parakeet v2/v3/110M, nemotron streaming, parakeet-unified, canary-180m-flash, qwen3-asr-0.6B, moonshine v1, Whisper) and which do not (canary-1b-flash, Granite, Kyutai, Voxtral, Moonshine v2).
- [S0325] sherpa-onnx pre-trained models index — k2-fsa, undated. https://k2-fsa.github.io/sherpa/onnx/pretrained_models/index.html. Accessed 2026-10-24. Supports: model families supported (NeMo transducer/CTC incl. Parakeet TDT, Whisper, Moonshine, Qwen3-ASR, Cohere Transcribe…).

- [S0326] sherpa-onnx Qwen3-ASR documentation — k2-fsa, undated. https://k2-fsa.github.io/sherpa/onnx/qwen3-asr/index.html. Accessed 2026-10-24. Supports: Qwen3-ASR-0.6B int8 on Android (arm64-v8a, armv7, x86, x86_64) and iOS.
- [S0327] Hugging Face Hub model API metadata — Hugging Face, queried 2026-10-24. https://huggingface.co/api/models/ (one per model in the table). Accessed 2026-10-24. Supports: safetensors parameter totals (e.g. Qwen3-ASR-0.6B 938M, granite-4.0-1b-speech 2.31B, nemotron 618M), licence tags, repo creation dates.
- [S0328] Tokenizer round-trip test of published tokenizer.json / vocab.json files — own test with HF `tokenizers`, 2026-10-24. https://huggingface.co parakeet-tdt-0.6b-v3, nemotron-speech-streaming-en-0.6b, whisper-small, moonshine-base, moonshine-streaming-medium, granite-speech-5.0-470m-turboctc, granite-4.0-1b-speech, kyutai/stt-1b-en_fr-trfs; Qwen3-ASR-0.6B vocab.json. Accessed 2026-10-24. Supports: vocab sizes and whether “” survives encode/decode.
- [S0329] End-to-End Speech Recognition and Disfluency Removal — Jamshid Lou & Johnson / arXiv 2009.10298 (Findings of EMNLP 2020), 2020-09-22. https://arxiv.org/abs/2009.10298. Accessed 2026-10-24. Supports: CTC vs LSTM-seq2seq vs Transformer trained to output fluent transcripts; Transformer best on disfluency removal (DER 24.0 vs CTC 26.2); E2E trails the pipeline.
- [S0330] Streaming Joint Speech Recognition and Disfluency Detection — Futami et al. / arXiv 2211.08726 (ICASSP 2023), 2022-11. https://arxiv.org/abs/2211.08726. Accessed 2026-10-24. Supports: Transformer encoder-decoder joint models beat a BERT pipeline; inline disfluency tags cause latency problems in streaming.
- [S0331] English-Hebrew-Codeswitching-Parakeet-Finetune README — Daniel Rosehill (local repo), 2026-10-22. ~/repos/github/audio-dictation/English-Hebrew-Codeswitching-Parakeet-Finetune/README.md. Accessed 2026-10-24. Supports: owner found v2 more accurate than v3 for English dictation; owner has fine-tuned Nemotron streaming, Parakeet 110M, Qwen3-ASR-1.7B, canary-qwen-2.5b, asr4all.

6.5 05 datasets

- [S0501] amaai-lab/DisfluencySpeech dataset card + API — AMAAI Lab / Hugging Face, lastModified 2024-06-27. https://huggingface.co/datasets/amaai-lab/DisfluencySpeech (README via /raw/main/README.md; https://huggingface.co/api/datasets/amaai-lab/DisfluencySpeech). Accessed 2026-10-24. Supports: apache-2.0; 4,500/250/250 splits; transcript_annotated/_a/_b/_c tiers (b removes filled pauses, editing terms, discourse markers; c also false starts); single speaker re-creating ~10 h of Switchboard.
- [S0502] nyralabs/disfluency_speech_english dataset card + rows — Nyra Health / Hugging Face, undated. https://huggingface.co/datasets/nyralabs/disfluency_speech_english (rows via datasets-server /rows). Accessed 2026-10-24. Supports: apache-2.0; derived from DisfluencySpeech; 4,957 utts, ~9.4 h; verbatim_transcript vs intended_transcript pairs.
- [S0503] pradachan/auto-whisper-disfluency API metadata — pradachan / Hugging Face, created 2026-06-09, lastModified 2026-06-15. https://huggingface.co/api/datasets/pradachan/auto-whisper-disfluency (README via /raw/main/README.md). Accessed 2026-10-24. Supports: no license field; splits train 18,628 / val 2,328 / test 2,329 / probe 250; fields input_features, labels, eval_reference, bucket, audio; 82.8 GB.
- [S0504] kensho/spgispeech dataset page and gated Terms of Usage — Kensho / Hugging Face, lastModified 2026-01-20. https://huggingface.co/datasets/kensho/spgispeech. Accessed 2026-10-24. Supports: 5,000 h, ~50k speakers, fully formatted; license “other”; “academic research purposes and internal use only”; no redistribution, no derivative works without written consent.

- [S0505] LibriSpeech-PC (OpenSLR 145) and paper arXiv 2310.02943 — Meister et al. / OpenSLR, 2023. https://openslr.org/145/ ; https://arxiv.org/abs/2310.02943. Accessed 2026-10-24 (search result summary, not read in full). Supports: CC BY 4.0, JSON manifests only (audio from SLR12), PnC restored from book text, ~90% of hours retained (train-other-500 449.10 h, train-clean-360 333.13 h, train-clean-100 91.61 h).
- [S0506] openslr/librispeech_asr dataset card — OpenSLR / Hugging Face, lastModified 2025-07-25. https://huggingface.co/datasets/openslr/librispeech_asr. Accessed 2026-10-24. Supports: ~1,000 h read audiobook speech; CC BY 4.0.
- [S0507] edinburghcstr/ami dataset card + rows; AMI Meeting Corpus licence page — University of Edinburgh CSTR, card lastModified 2026-01-13. https://huggingface.co/datasets/edinburghcstr/ami ; https://groups.inf.ed.ac.uk/ami/corpus/license.shtml. Accessed 2026-10-24. Supports: 100 h meetings, mostly non-native; Kaldi s5 normalised uppercase text with fillers (“MM-HMM”); CC BY 4.0.
- [S0508] JacobLinCool/ami-disfluent API metadata + rows — Hugging Face, undated. https://huggingface.co/datasets/JacobLinCool/ami-disfluent. Accessed 2026-10-24. Supports: cc-by-4.0; 20,018/2,483/1,892 utts; verbatim text with UM/UH and repeats.
- [S0509] Welcome to the ICSI Corpus — University of Edinburgh, undated. https://groups.inf.ed.ac.uk/ami/icsi/. Accessed 2026-10-24. Supports: ~70 h meetings, close-talk mics; orthographic transcription, dialog acts; signals + transcription released under CC BY 4.0.
- [S0510] MLCommons/peoples_speech dataset card + rows — MLCommons / Hugging Face, lastModified 2024-11-20. https://huggingface.co/datasets/MLCommons/peoples_speech. Accessed 2026-10-24. Supports: 30,000+ h; CC-BY and CC-BY-SA subsets; some training transcripts likely ASR output; lowercase unpunctuated spoken-form text with repeats (“i i i”).
- [S0511] speechcolab/gigaspeech gated terms + SpeechColab/GigaSpeech README — SpeechColab, card lastModified 2026-02-07. https://huggingface.co/datasets/speechcolab/gigaspeech ; https://github.com/SpeechColab/GigaSpeech. Accessed 2026-10-24. Supports: 10,000 h labelled / 33,000 h total; SpeechColab does not own audio copyright; “non-commercial research and educational purposes”; text_tn uppercase spoken form with etc.; conversational fillers present.
- [S0512] mozilla-foundation/common_voice_17_0 README; Mozilla Data Collective dataset pages — Mozilla, 2025. https://huggingface.co/datasets/mozilla-foundation/common_voice_17_0 ; https://dev.datacollective.mozillafoundation.org/datasets/cmflnn4815p33e4fb0591ef4y. Accessed 2026-10-24 (MDC page from search result summary only). Supports: from Oct 2025 Common Voice only via MDC; CC0-1.0 label plus “You agree that you will not re-host or re-share this dataset”.
- [S0513] revdotcom/speech-datasets README, earnings21/README.md, earnings22/README.md, multireferences/README.md — Rev, release notes to 202408. https://github.com/revdotcom/speech-datasets. Accessed 2026-10-24. Supports: Earnings-21 39 h / 44 files, Earnings-22 119 h / 125 files; CC BY-SA 4.0 badge linking LICENSE.md; .nlp token files with entity tags; multireferences hold two Rev transcript styles per file (subfolders not reachable via raw).
- [S0514] Earnings-21: A Practical Benchmark for ASR in the Wild (Del Rio et al.) — Interspeech 2021 / arXiv 2104.11348, 2021-04. https://ar5iv.labs.arxiv.org/html/2104.11348. Accessed 2026-10-24. Supports: Rev human transcription, “verbatim” incl. filler words and false starts; normalisation tags (cardinals, ordinals, YEAR); audio under fair use footnote, no data licence named in the paper.
- [S0515] distil-whisper/earnings22 card + rows — Hugging Face, lastModified 2023-10-13. https://huggingface.co/datasets/distil-whisper/earnings22. Accessed 2026-10-24. Supports: chunked test 57,391 segments; cased/punctuated transcription; no license field.

- [S0516] Switchboard-1 Release 2 (LDC97S62) — Linguistic Data Consortium, 1997. https://catalog.ldc.upenn.edu/LDC97S62. Accessed 2026-10-24. Supports: ~260 h, 543 speakers, ~2,400 conversations; “LDC User Agreement for Non-Members”.
- [S0517] Fisher English Training Speech Part 1 Speech (LDC2004S13) — LDC, 2004. https://catalog.ldc.upenn.edu/LDC2004S13. Accessed 2026-10-24. Supports: 984 h CTS; transcripts in LDC2004T19; LDC non-member agreement.
- [S0518] LDC licensing (data management: using) — LDC, undated. https://www.ldc.upenn.edu/data-management/using/licensing. Accessed 2026-10-24. Supports: non-member/non-profit licensees “noncommercial linguistic research and education only”, “cannot use LDC data to develop or test products for commercialization”; corpus-specific licences supersede; no statement on trained models.
- [S0519] NXT Switchboard Annotations (LDC2009T26) and Edinburgh NXT Switchboard pages — LDC / University of Edinburgh, 2009-11-20. https://catalog.ldc.upenn.edu/LDC2009T26 ; https://groups.inf.ed.ac.uk/switchboard/download.html. Accessed 2026-10-24 (search result summary only, not read in full). Supports: CC BY-NC-SA 3.0 for non-members; disfluency layer from Penn Treebank (650 files), Treebank-3 is LDC99T42.
- [S0520] WhispSynth: Scaling Multilingual Whisper Corpus through Real Data Curation and A Novel Pitch-free Generative Framework (Tan et al.) — arXiv 2603.14853v1, 2026-03. https://arxiv.org/pdf/2603.14853 (PDF read locally via pdftotext); GitHub API https://api.github.com/repos/tan90xx/cosywhisper/contents. Accessed 2026-10-24. Supports: Table 1 whisper corpora with hours/speakers/licence (wTIMIT 29.44 h, 48 spk, N.A.; CHAINs 2.55 h, 36 spk, BY-ND; EARs 3.22 h, BY-NC; Expresso 1.32 h, BY-NC; wSPIRE 18 h, N.A.); WhispSynth ≈118 h en+zh, 479 spk, MIT; repo empty on 2026-10-24.
- [S0521] CHAINS corpus: LINGUIST List 17.2363 announcement; LDC2008S09 catalogue — Cummins et al. / UCD / LDC, 2006/2008. https://linguistlist.org/issues/17/2363/ ; https://catalog.ldc.upenn.edu/LDC2008S09. Accessed 2026-10-24 (search result summary only; chains.ucd.ie ECONNREFUSED). Supports: original CC licence free for commercial and non-commercial research, share-alike redistribution; LDC lists non-member agreement; 36 speakers incl. whisper condition.
- [S0522] wTIMIT description in Chang et al. (S0410) and WhispSynth (S0520) — 2020/2026. https://arxiv.org/abs/2005.01972. Accessed 2026-10-24. Supports: ~26 h whispered + ~26 h normal, 48 US/Singapore speakers, 450 sentences; no public licence located.
- [S0523] facebookresearch/ears_dataset README — Meta / Richter et al., Interspeech 2024. https://github.com/facebookresearch/ears_dataset. Accessed 2026-10-24. Supports: 100 h, 107 speakers, 7 reading styles incl. whisper; “code and dataset are released under CC-NC 4.0”.
- [S0524] ylacombe/expresso dataset card — Hugging Face (Nguyen et al. 2023), undated. https://huggingface.co/datasets/ylacombe/expresso. Accessed 2026-10-24. Supports: 40 h, 4 speakers; whisper 79 min read + 86 min improvised (2.8 h); CC BY-NC 4.0.
- [S0525] neilshahnms/Claris-Whispered-English API + file tree — Hugging Face, created 2026-06-11. https://huggingface.co/api/datasets/neilshahnms/Claris-Whispered-English. Accessed 2026-10-24. Supports: 3,365 rows, audiofolder wav + txt, file names “english_nonnative_effort”; no licence, no README.
- [S0526] Malfaro43/whisperedAudio-benchmark card — Hugging Face, undated. https://huggingface.co/datasets/Malfaro43/whisperedAudio-benchmark. Accessed 2026-10-24. Supports: EARS + Expresso normal vs whisper, background-speaker mixes at 10%/25%; no licence on card.
- [S0527] LibriStutter: review arXiv 2107.04057; HF mirror stillerman/libristutter-4.7k — Sheikh et al. 2021 / Hugging Face. https://arxiv.org/pdf/2107.04057 ; https://huggingface.co/api/datasets/stillerman/libristutter-4.7k. Accessed 2026-10-24 (review from search summary only). Supports:

synthetic stutters on LibriSpeech, ~20 h/50 spk vs 100 h/10 spk disagreement; mirror 4,262/474 rows, no licence.
- [S0528] LLM-Dys README and arXiv 2505.22029; tong0/LLM_Dys API — Berkeley Speech Group, 2025-05. https://github.com/Berkeley-Speech-Group/LLM-Dys ; https://arxiv.org/abs/2505.22029 ; https://huggingface.co/api/datasets/tong0/LLM_Dys. Accessed 2026-10-24. Supports: ~12,790 h (~5 TB) generate-yourself, VCTK voices, VITS/E2-TTS; 11 word/phoneme dysfluency types; no dataset licence on repo or HF.
- [S0529] SSDM (arXiv 2408.16221) and SSDM 2.0 (arXiv 2412.00265) — Lian et al., 2024. https://arxiv.org/pdf/2408.16221. Accessed 2026-10-24 (search result summary only). Supports: Libri-Dys and Libri-Co-Dys built from LibriTTS with StyleTTS2, phoneme-level injection; no licence stated.
- [S0530] apple-aiml-research/ml-stuttering-events-dataset LICENSE; th-nuernberg SEP-28k-E README — Apple / TH Nürnberg, 2021. https://github.com/apple-aiml-research/ml-stuttering-events-dataset. Accessed 2026-10-24. Supports: Apple sample-code licence on the label release; audio not distributed (URLs only), copyright with podcast owners (search summary).
- [S0531] nvidia/Granary dataset card — NVIDIA / Hugging Face, lastModified 2026-06-26. https://huggingface.co/datasets/nvidia/Granary. Accessed 2026-10-24. Supports: ~643k h ASR across 25 languages; LibriLight ~23.5k h EN; PnC restored with Qwen-2.5-7B; CC BY 4.0 card, YODAS-Granary CC-BY-3.0, source audio per original licence.
- [S0532] facebook/voxpopuli dataset card — Meta / Hugging Face, lastModified 2026-01-30. https://huggingface.co/datasets/facebook/voxpopuli. Accessed 2026-10-24. Supports: English 543 h, 1,313 speakers; raw_text and normalized_text; CC0 (plus EP legal notice).
- [S0533] k2-fsa/libriheavy README + GitHub licence — k2-fsa, 2023 (arXiv 2309.08105). https://github.com/k2-fsa/libriheavy. Accessed 2026-10-24. Supports: 50,000 h from LibriLight; book transcript with casing/punctuation plus pre_texts context; Apache-2.0 repo licence.
- [S0534] Hebrish datasets listing and READMEs (Disfluent-Read, Self-Correction, Spoken-Aside, Repeated-Word, Currency, Weird-Voice) — Daniel Rosehill / Hugging Face, lastModified 2026-10-22 to 2026-10-24. https://huggingface.co/api/datasets?author=Hebrish ; https://huggingface.co/datasets/Hebrish/Self-Correction-Sentences. Accessed 2026-10-24. Supports: CC BY 4.0; single speaker, phone mic; field schemes (text vs spoken_text/verbatim_text, aside_text/aside_position, currency written forms); Disfluent-Read 51/4 rows.
- [S0535] tatianapassali/artificial-disfluency-generation (LARD) LICENSE.txt — Passali et al., LREC 2022, last push 2022-09-28. https://github.com/tatianapassali/artificial-disfluency-generation. Accessed 2026-10-24. Supports: CC BY-NC-SA 4.0; generates repetitions, restarts, replacements.
- [S0536] stillerman/fdt-disfluency-synthetic card — Hugging Face, undated. https://huggingface.co/datasets/stillerman/fdt-disfluency-synthetic. Accessed 2026-10-24. Supports: CC BY-NC-SA 4.0 (inherits DailyDialog); injected fillers, repeats, false starts, substitution repairs, KEEP/DELETE tags.
- [S0537] google-research-datasets/Disfl-QA README — Google, 2021. https://github.com/google-research-datasets/disfl-qa. Accessed 2026-10-24. Supports: ~12k disfluent questions, >90% corrections or restarts; CC BY 4.0.
- [S0538] NVIDIA/NeMo-text-processing — NVIDIA, last push 2026-10-24. https://github.com/NVIDIA/NeMo-text-processing. Accessed 2026-10-24. Supports: WFST text normalisation / ITN for ASR and TTS; Apache-2.0.
- [S0539] rwsproat/text-normalization-data README — Sproat, undated. https://github.com/rwsproat/text-normalization-data. Accessed 2026-10-24. Supports: data hosted on Kaggle (text-normalization-for-english-russian-and-polish); no licence in repo.
- [S0540] hexgrad/Kokoro-82M and CSTR-Edinburgh/vctk API metadata — Hugging Face. https://huggingface.co/api/models/hexgrad/Kokoro-82M ; https://huggingface.co/api/datasets/CSTR-Edinburgh/vctk. Accessed 2026-10-24. Supports: Kokoro apache-2.0; VCTK cc-by-4.0.

- [S0541] Device-directed speech corpora search: SAS (arXiv 2604.08412), Modality Dropout DDSD (arXiv 2310.15261) — 2023/2026. https://arxiv.org/pdf/2310.15261 ; https://arxiv.org/pdf/2604.08412. Accessed 2026-10-24 (search result summary only). Supports: 530 h / 1,300-participant DDSD set without stated release; SAS 600 h proprietary; no public English addressee corpus found.
- [S0542] HF dataset keyword searches (disfluen, whispered, stutter, self-correction, itn, switchboard, fisher) — Hugging Face API, 2026-10-24. https://huggingface.co/api/datasets?search=disfluen&sort=downloads. Accessed 2026-10-24. Supports: candidate discovery; gated hhoangphuoc/ami-disfluency (apache-2.0 tag), ltrciiith/bhasik_Indian_English_disfluency_corpus (cc-by-nd-4.0 tag); unofficial Switchboard/Fisher uploads (e.g. dys-asr/switchboard-sr8000, kalbin/fisher-sft-v2).